POSIX THREADS ON LINUX

Linux Pthreads

Theory first, then small staged examples up to mutexes. Every example is a program you can build and run.

PLAN

What we cover

Part	Topics	Examples
Theory	Multithreading models, Pthreads, Win32 threads, threading issues, thread pools, Linux threads	Slides only
1 Foundations	What threads share and what they keep	p1a, p1b
2 Lifecycle	Create, arguments, results, exit, detach	p2a to p2e
3 Attributes	Stack size, detach state	p3a, p3b
4 Mutexes	Data races, lock and unlock, trylock, init and destroy	p4a to p4d

SETUP

Build and run the examples

ALL EXAMPLES

```
cd pthreads-class
make
./p1a_hello
```

ONE FILE BY HAND

```
gcc -Wall -g -O0 -pthread \
  -o p2a_args p2a_args.c
```

NOTES

- Always pass -pthread, at compile and link time.
- make p2a_args builds just that program.
- -O0 keeps the race in Part 4 easy to see.
- The advanced folder holds larger programs for later topics.

THEORY

Before the code

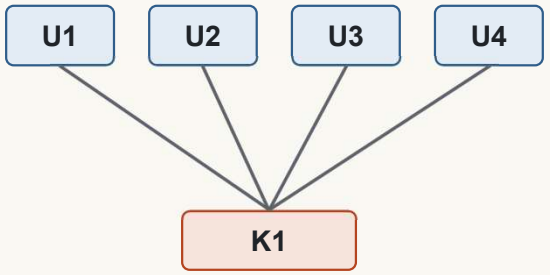
Multithreading models, Pthreads, Win32 threads, threading issues, thread pools, and Linux threads

THEORY · MULTITHREADING MODELS

User threads on kernel threads

Many-to-one

User threads

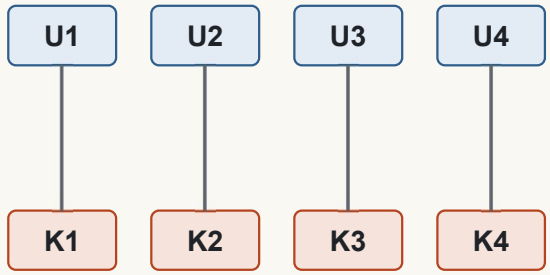


Kernel threads

Many user threads share one kernel thread. A blocking call stops all of them, and only one CPU is used.

One-to-one

User threads

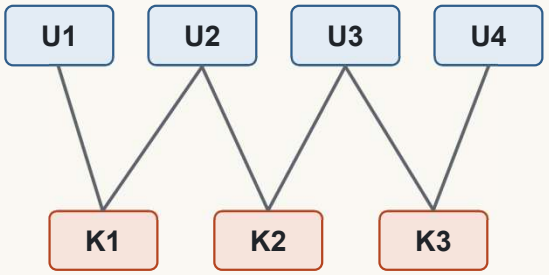


Kernel threads

Each user thread gets its own kernel thread. Full parallelism, but every thread costs kernel resources. Linux and Windows work this way.

Many-to-many

User threads



Kernel threads

Many user threads over a smaller number of kernel threads. Flexible, but hard to build and to schedule well.

THEORY · MULTITHREADING MODELS

Comparing the models

Model	Mapping	Parallel	A thread blocks	Seen in
Many-to-one	N user : 1 kernel	No	All threads stop	Early green-thread libraries
One-to-one	1 user : 1 kernel	Yes	Only that thread	Linux, Windows, macOS
Many-to-many	N user : M kernel	Yes	Others keep running	Older Solaris, Go runtime
Two-level	M:N, plus 1:1 on request	Yes	Others keep running	IRIX, HP-UX, Tru64

Pthreads does not fix the model. On Linux each pthread_create makes a kernel thread, so the one-to-one row applies to everything in this course.

THEORY · PTHREADS

The POSIX thread API

- POSIX.1c: a standard API, not an implementation
- Declared in pthread.h, linked with -pthread
- Native on Linux, macOS, FreeBSD and Solaris
- No native version on Windows, ports exist
- Calls return 0 on success, or an error number

API group	Example calls
Thread management	pthread_create, pthread_join, pthread_exit
Attributes	pthread_attr_init, pthread_attr_setstacksize
Mutexes	pthread_mutex_lock, pthread_mutex_unlock
Condition variables	pthread_cond_wait, pthread_cond_signal
Read-write locks	pthread_rwlock_rdlock, pthread_rwlock_wrlock
Thread-specific data	pthread_key_create, pthread_getspecific

This course covers the first three groups.

THEORY · WIN32 THREADS

The Windows thread API

- Threads are kernel objects, named by a HANDLE
- One-to-one model, like Linux
- Thread function: DWORD WINAPI fn(LPVOID)
- In C, prefer _beginthreadex so the C runtime sets up its per-thread state
- Close every handle with CloseHandle

Task	Pthreads	Win32
Create	pthread_create	CreateThread, _beginthreadex
Wait for end	pthread_join	WaitForSingleObject
End thread	pthread_exit	ExitThread
Thread ID	pthread_self	GetCurrentThreadId
Mutex	pthread_mutex_t	CreateMutex, CRITICAL_SECTION
Lock, unlock	pthread_mutex_lock, unlock	WaitForSingleObject, ReleaseMutex

THEORY · WIN32 THREADS

The same program in Pthreads and in Win32

Pthreads (Linux): p1a_hello.c

```c
#include <pthread.h>
#include <stdio.h>

/* A thread function takes one void * and returns a void *. */
static void *hello(void *arg)
{
    (void)arg;
    printf("Hello from the new thread\n");
    return NULL;
}

int main(void)
{
    pthread_t tid;

    /* Start hello() in a new thread: default attributes, no arg. */
    pthread_create(&tid, NULL, hello, NULL);

    /* Wait for it to finish before main returns. */
    pthread_join(tid, NULL);

    printf("Thread finished, main exits\n");
    return 0;
}
```

Win32 (Windows): win32_hello.c

```c
#include <windows.h>
#include <stdio.h>

/* Thread function: DWORD WINAPI, one LPVOID argument. */
DWORD WINAPI hello(LPVOID arg)
{
    (void)arg;
    printf("Hello from a Win32 thread\n");
    return 0;
}

int main(void)
{
    DWORD id;

    /* Args: security, stack size, function, argument, flags, ID. */
    HANDLE h = CreateThread(NULL, 0, hello, NULL, 0, &id);

    /* Block until the thread ends. */
    WaitForSingleObject(h, INFINITE);

    /* Release the thread's kernel object. */
    CloseHandle(h);
    return 0;
}
```

Same shape in both: create the thread, wait for it, clean up. Only the names and the handle type differ.

THEORY · THREADING ISSUES

What threads make harder

fork() and exec()

Does fork() copy every thread? POSIX copies only the calling thread. exec() replaces the whole process, threads included.

Signal handling

A synchronous signal, such as a bad memory access, goes to the faulting thread. An asynchronous one, such as Ctrl-C, goes to any thread that has not blocked it.

Thread cancellation

Asynchronous: the target stops at once. Deferred: it stops at a safe cancellation point. Pthreads defaults to deferred.

Thread-local storage

Data private to one thread but kept across function calls, such as errno. A local variable cannot do that.

Scheduler activations

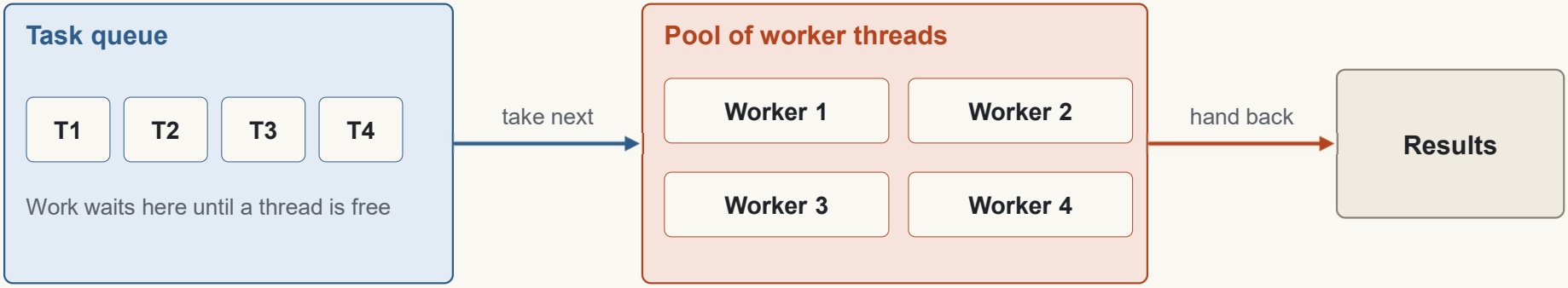
How the kernel and a user-level thread library cooperate in many-to-many models: the kernel notifies the library when a thread blocks.

Shared data

Data races and deadlock appear as soon as threads share memory. Part 4 starts with races and mutexes.

THEORY · THREAD POOLS

Create threads once, reuse them



- No thread creation cost per request: the same threads run task after task
- A fixed number of threads limits CPU and memory use
- The task is separate from how it runs: it can be queued, delayed or repeated
- Pool size: about the number of cores for compute work, more for work that waits on I/O

THEORY · LINUX THREADS

Threads are tasks made by clone()

- The kernel schedules one kind of thing: the task
- A process and a thread are both tasks, created by clone()
- pthread_create calls clone() with flags that share resources
- One-to-one model; the library is NPTL, part of glibc
- getpid() is per process, gettid() is per thread; ps -L and top -H list threads

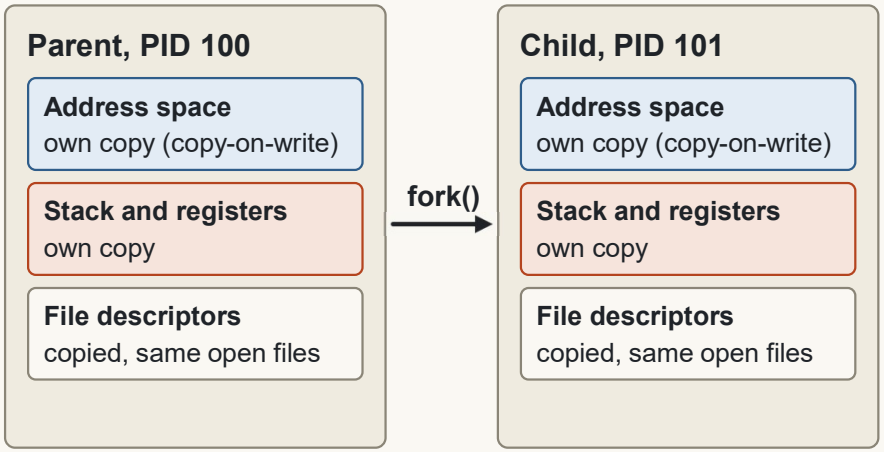
clone() flag	What is shared
CLONE_VM	The address space
CLONE_FS	Working directory, root
CLONE_FILES	Open file descriptors
CLONE_SIGHAND	Signal handlers
CLONE_THREAD	Same process ID and thread group

fork() is a clone() without these flags, so the child gets its own copies.

THEORY · FORK() AND PTHREAD_CREATE()

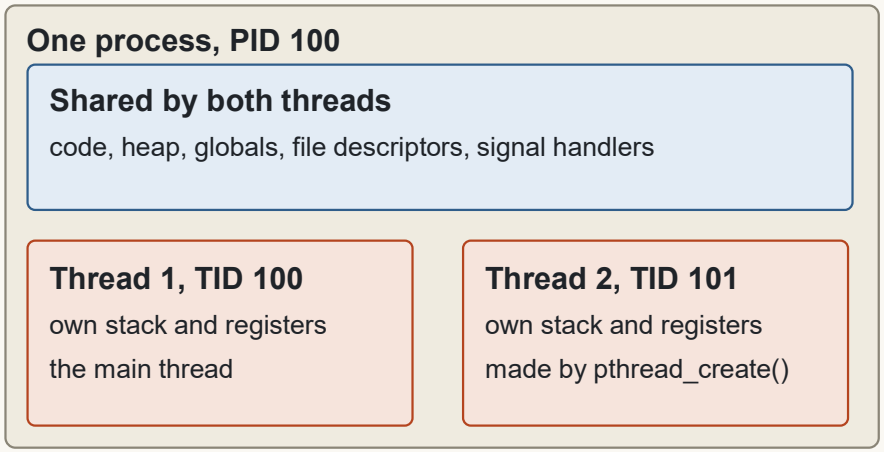
A new process, or a new thread

fork(): two processes



A new process with its own address space. Memory is copied lazily, so a write in one process is not seen by the other.

pthread_create(): two threads

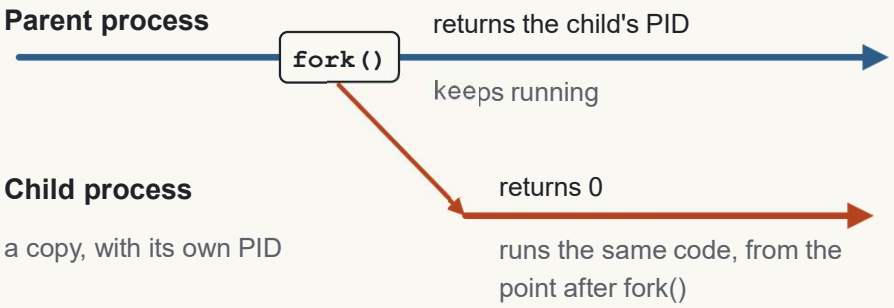


A new thread inside the same process. It gets its own stack; everything else is shared, so a write by one thread is seen by the other.

THEORY · FORK() AND PTHREAD_CREATE()

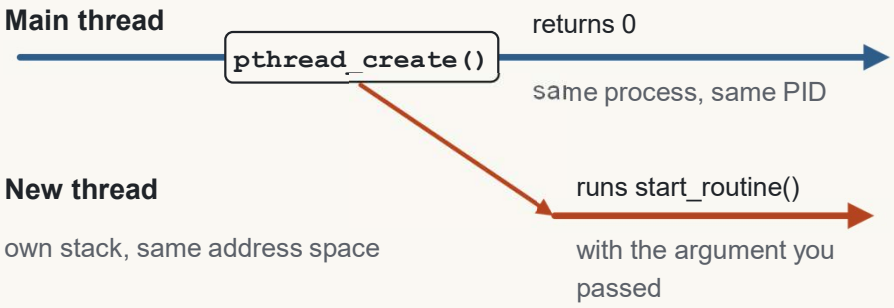
What happens after the call

fork() returns twice



Once in each process. The child carries on from the same point in the same code, and tells itself apart by the return value.

pthread_create() returns once



Once, in the creating thread. The new thread begins at the function you give it, not after the call.

What differs

	fork()	pthread_create()
Creates	A new process	A new thread in the same process
Address space	Separate copy (copy-on-write)	Shared
IDs	New PID	Same PID, new thread ID
File descriptors	Copied; both refer to the same open files	One shared table
Communication	Pipes, sockets, shared memory	Plain variables, with locks
Return value	Twice: 0 in the child, the child's PID in the parent	Once: 0 on success, thread ID through a pointer
Cost	Heavier: its own page tables and process structures	Lighter: a stack and a task
If it crashes	The other process keeps running	The whole process dies
Under the hood	clone() without the sharing flags	clone() with CLONE_VM, CLONE_FILES, CLONE_THREAD and others

PART 1

Foundations

What threads share and what they keep

PART 1 · FOUNDATIONS

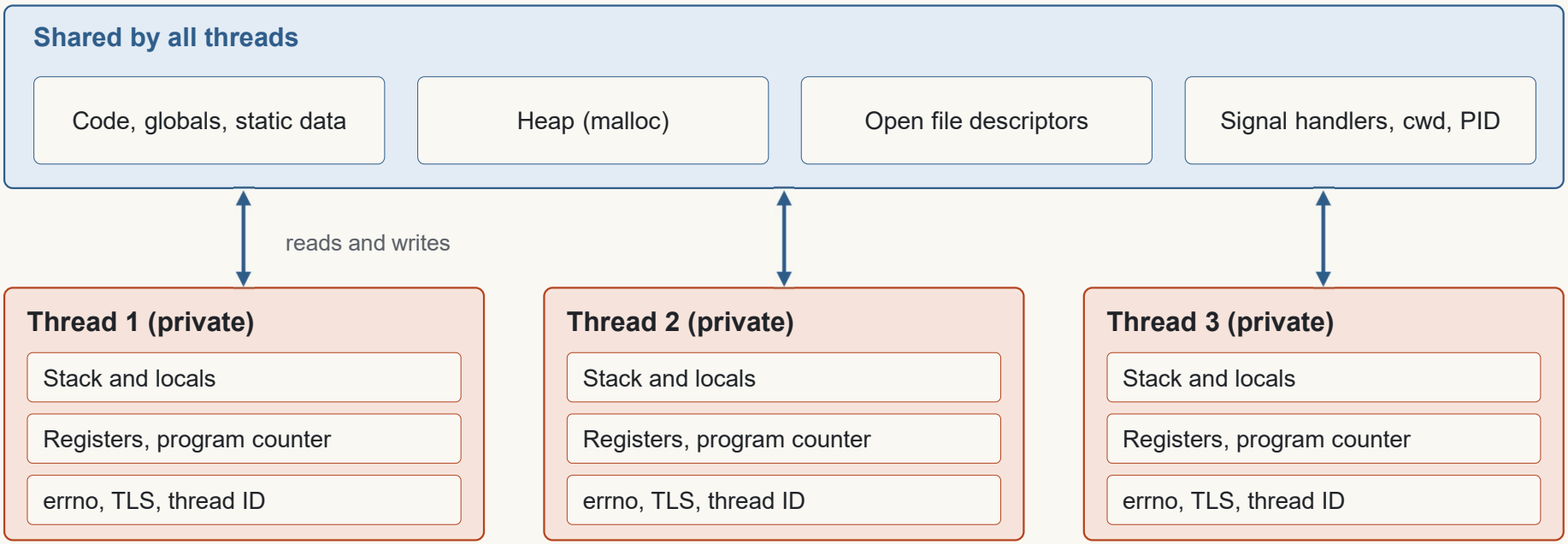
What threads share, and what they keep

Shared by every thread	Private to each thread
Code, globals and static data	Stack and local variables
Heap memory from malloc	Registers and program counter
Open file descriptors	Thread ID
Signal handlers, working directory	errno and signal mask
Process ID	Thread-local storage

Sharing is the whole point of threads, and also the source of every bug in this class. Any write to shared data needs a plan.

PART 1 · FOUNDATIONS

One process: shared memory, private state



PART 1 · FOUNDATIONS · EXAMPLE 1 OF 2

Hello, thread

Highlighted: pthread_create, pthread_join

```c
/* p1a_hello.c - the smallest threaded program.
   Build: gcc -Wall -pthread -o p1a_hello p1a_hello.c */
#include <pthread.h>
#include <stdio.h>

/* A thread function takes one void * and returns a void *. */
static void *hello(void *arg)
{
    (void)arg;
    printf("Hello from the new thread\n");
    return NULL;
}
```

```c
int main(void)
{
    pthread_t tid;

    /* Start hello() in a new thread: default attributes, no arg. */
    pthread_create(&tid, NULL, hello, NULL);

    /* Wait for it to finish before main returns. */
    pthread_join(tid, NULL);

    printf("Thread finished, main exits\n");
    return 0;
}
```

```
$ make p1a_hello
$ ./p1a_hello
Hello from the new thread
Thread finished, main exits
```

WHAT TO LOOK FOR

- pthread_create starts hello() in a new thread
- pthread_join waits, so main does not exit first
- Build with -pthread (the Makefile already does)

PART 1 · FOUNDATIONS · EXAMPLE 2 OF 2

What is shared, what is private

Highlighted: getpid, pthread_self

```c
/* p1b_shared_private.c - shared versus private data.
   Build: make p1b_shared_private */
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

/* One copy exists, so every thread sees the same variable. */
int shared = 100;

static void *worker(void *arg)
{
    /* A local variable lives on this thread's own stack. */
    int mine = 1;

    /* getpid(): same process ID in every thread. */
    pid_t pid = getpid();

    /* pthread_self(): a different thread ID in each thread. */
    pthread_t me = pthread_self();
    (void)arg;
    printf("pid=%d thread=%lu\n", pid, (unsigned long)me);
    printf("  &shared=%p  &mine=%p\n",
           (void *)&shared, (void *)&mine);

    /* Stay alive so both threads exist at the same time. */
    sleep(1);
    return NULL;
}

int main(void)
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

```
$ make p1b_shared_private
$ ./p1b_shared_private
pid=621 thread=139826869302976
  &shared=0x559511b30010  &mine=0x7f2bfadfeea8
pid=621 thread=139826860910272
  &shared=0x559511b30010  &mine=0x7f2bfa5fdea8
```

WHAT TO LOOK FOR

- Same pid in both threads: it is one process
- Different thread IDs: two threads
- &shared is identical, &mine is different

PART 2

Thread lifecycle

Create, pass data, get results, exit, detach

PART 2 · LIFECYCLE · EXAMPLE 1 OF 5

Passing an argument

Highlighted: pthread_create (4th argument)

```c
/* p2a_args.c - give each thread its own input.
   Build: gcc -Wall -pthread -o p2a_args p2a_args.c */
#include <pthread.h>
#include <stdio.h>

#define N 3

/* The 4th argument of pthread_create arrives here as arg. */
static void *worker(void *arg)
{
    /* Cast the void * back to the type we passed in. */
    int id = *(int *)arg;

    printf("worker got id %d\n", id);
    return NULL;
}
```

```c
int main(void)
{
    pthread_t tid[N];

    /* One slot per thread, so no two threads share an argument. */
    int ids[N];

    for (int i = 0; i < N; i++) {
        ids[i] = i;

        /* 4th argument: the address of this thread's own slot. */
        pthread_create(&tid[i], NULL, worker, &ids[i]);
    }
    for (int i = 0; i < N; i++)
        pthread_join(tid[i], NULL);
    return 0;
}
```

```
$ make p2a_args
$ ./p2a_args
worker got id 0
worker got id 1
worker got id 2
```

WHAT TO LOOK FOR

- The 4th argument of pthread_create becomes arg
- A void * can point at any type: cast it back
- One slot per thread, so no argument is shared

PART 2 · LIFECYCLE · EXAMPLE 2 OF 5

Returning a result

Highlighted: pthread_join (2nd argument)

```c
/* p2b_return.c - get a result back from a thread.
   Build: gcc -Wall -pthread -o p2b_return p2b_return.c */
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

static void *square(void *arg)
{
    int n = *(int *)arg;

    /* The result must outlive the thread, so put it on the heap. */
    int *result = malloc(sizeof *result);
    *result = n * n;

    /* Returning a pointer hands it to whoever joins us. */
    return result;
}
```

```c
int main(void)
{
    pthread_t tid;
    int input = 7;
    void *ret;

    pthread_create(&tid, NULL, square, &input);

    /* 2nd arg of pthread_join receives the returned pointer. */
    pthread_join(tid, &ret);

    printf("7 squared = %d\n", *(int *)ret);
    free(ret);
    return 0;
}
```

```
$ make p2b_return
$ ./p2b_return
7 squared = 49
```

WHAT TO LOOK FOR

- The pointer the thread returns lands in ret
- Heap memory outlives the thread, a local would not
- The joiner owns the result and frees it

PART 2 · LIFECYCLE · EXAMPLE 3 OF 5

Ending a thread early

Highlighted: pthread_exit

```c
/* p2c_exit.c - end a thread early with pthread_exit.
   Build: gcc -Wall -pthread -o p2c_exit p2c_exit.c */
#include <pthread.h>
#include <stdio.h>

static void helper(void)
{
    /* Ends the whole calling thread, not just this function. */
    pthread_exit((void *)42L);
}

static void *worker(void *arg)
{
    (void)arg;
    printf("worker: calling helper\n");
    helper();
    /* Never reached: pthread_exit does not return. */
    printf("worker: you will not see this\n");
    return NULL;
}

int main(void)
{
    pthread_t tid;
    void *ret;

    pthread_create(&tid, NULL, worker, NULL);
    pthread_join(tid, &ret);
    printf("main: thread exited with %ld\n", (long)ret);
    return 0;
}
```

```
$ make p2c_exit
$ ./p2c_exit
worker: calling helper
main: thread exited with 42
```

WHAT TO LOOK FOR

- Ends the calling thread, even from a helper function
- It never returns to its caller
- The value reaches pthread_join, as in the last example

PART 2 · LIFECYCLE · EXAMPLE 4 OF 5

A thread nobody joins

Highlighted: pthread_detach

```c
/* p2d_detach.c - a thread nobody will join.
   Build: gcc -Wall -pthread -o p2d_detach p2d_detach.c */
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

static void *background(void *arg)
{
    (void)arg;
    sleep(1);
    printf("background: done, resources freed automatically\n");
    return NULL;
}
```

```c
int main(void)
{
    pthread_t tid;

    pthread_create(&tid, NULL, background, NULL);

    /* After this, joining tid is not allowed. The system frees
       the thread's resources by itself when it ends. */
    pthread_detach(tid);

    printf("main: not waiting for it\n");

    /* Demo only: keep the process alive long enough to see it. */
    sleep(2);
    return 0;
}
```

```
$ make p2d_detach
$ ./p2d_detach
main: not waiting for it
background: done, resources freed automatically
```

WHAT TO LOOK FOR

- A detached thread frees its own resources on exit
- Joining a detached thread is an error
- Use it for work whose result nobody needs

PART 2 · LIFECYCLE · EXAMPLE 5 OF 5

A classic mistake

Highlighted: &i, one variable shared by every thread

```c
/* p2e_loop_bug.c - the classic argument mistake.
   Build: gcc -Wall -pthread -o p2e_loop_bug p2e_loop_bug.c */
#include <pthread.h>
#include <stdio.h>

static void *worker(void *arg)
{
    /* Reads i later, after the loop may already have changed it. */
    int id = *(int *)arg;

    printf("worker sees id %d\n", id);
    return NULL;
}
```

```c
int main(void)
{
    pthread_t tid[3];
    int i;

    for (i = 0; i < 3; i++) {
        /* BUG: every thread gets the address of the same i. */
        pthread_create(&tid[i], NULL, worker, &i);
    }
    for (int k = 0; k < 3; k++)
        pthread_join(tid[k], NULL);

    /* Fix: pass &ids[i] from an array, as in p2a_args.c. */
    return 0;
}
```

```
$ make p2e_loop_bug
$ ./p2e_loop_bug
worker sees id 3
worker sees id 3
worker sees id 3
```

WHAT TO LOOK FOR

- All three threads get the address of the same i
- They read it later, after the loop has moved on
- The output changes from run to run: a race

PART 2 · LIFECYCLE

Ending a thread: join, exit, detach

Call	What happens
return v, or pthread_exit(v)	The thread ends; v goes to the joiner
pthread_join(t, &ret)	Waits, collects v, frees the thread's resources
pthread_detach(t)	Resources freed on exit; no join possible
pthread_self, pthread_equal	Own ID, and the safe way to compare IDs
main returns, or exit()	The whole process ends, every thread with it

Every thread should be joined or detached. Never neither, or its resources stay allocated.

PART 3

Thread attributes

Settings for new threads: stack size and detach state

PART 3 · ATTRIBUTES

What you can set with pthread_attr_t

Attribute	Call	Notes
Stack size	pthread_attr_setstacksize	Default follows ulimit -s, usually 8 MiB
Detach state	pthread_attr_setdetachstate	PTHREAD_CREATE_DETACHED or JOINABLE
Policy	pthread_attr_setschedpolicy	SCHED_OTHER, SCHED_FIFO, SCHED_RR
Priority	pthread_attr_setschedparam	Real-time priorities run 1 to 99
Inherit	pthread_attr_setinheritsched	Set EXPLICIT, or policy and priority are ignored
Affinity	pthread_attr_setaffinity_np	GNU extension; pins a thread to chosen cores

Real-time policies need root or an rtprio limit. Call pthread_attr_destroy when you are done with the object.

PART 3 · ATTRIBUTES · EXAMPLE 1 OF 2

Setting the stack size

Highlighted: setstacksize, getstacksize

```c
/* p3a_stack_size.c - choose a thread's stack size.
   Build: gcc -Wall -pthread -o p3a_stack_size p3a_stack_size.c */
#include <pthread.h>
#include <stdio.h>

static void *worker(void *arg)
{
    (void)arg;
    printf("worker running with the requested stack\n");
    return NULL;
}

int main(void)
{
    pthread_attr_t attr;
    pthread_t tid;
    size_t size;

    /* An attribute object holds settings for new threads. */
    pthread_attr_init(&attr);

    /* Ask for a 256 KiB stack (the default is usually 8 MiB). */
    pthread_attr_setstacksize(&attr, 256 * 1024);

    /* Read the value back to confirm it. */
    pthread_attr_getstacksize(&attr, &size);
    printf("stack size set to %zu KiB\n", size / 1024);

    /* Pass &attr as the 2nd argument instead of NULL. */
    pthread_create(&tid, &attr, worker, NULL);
    pthread_join(tid, NULL);
    pthread_attr_destroy(&attr);
    return 0;
}
```

```
$ make p3a_stack_size
$ ./p3a_stack_size
stack size set to 256 KiB
worker running with the requested stack
```

WHAT TO LOOK FOR

- pthread_attr_init prepares the attribute object
- setstacksize changes it, getstacksize reads it back
- Pass &attr as the 2nd argument of pthread_create

PART 3 · ATTRIBUTES · EXAMPLE 2 OF 2

Creating a detached thread

Highlighted: pthread_attr_setdetachstate

```c
/* p3b_detach_attr.c - create a thread that is already detached.
   Build: make p3b_detach_attr */
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

static void *worker(void *arg)
{
    (void)arg;
    printf("worker: detached from the start\n");
    return NULL;
}

int main(void)
{
    pthread_attr_t attr;
    pthread_t tid;

    pthread_attr_init(&attr);

    /* Like pthread_detach, but with no gap after pthread_create. */
    pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);

    pthread_create(&tid, &attr, worker, NULL);
    pthread_attr_destroy(&attr);

    /* Detached, so we cannot join. Sleep for the demo only. */
    sleep(1);
    return 0;
}
```

```
$ make p3b_detach_attr
$ ./p3b_detach_attr
worker: detached from the start
```

WHAT TO LOOK FOR

- Same effect as pthread_detach, set at creation
- Nothing to join, so main sleeps for the demo
- The attribute object can be destroyed right away

PART 4

Races and mutexes

Why shared data needs protection, and how a mutex provides it

PART 4 · MUTEXES

counter++ is three steps, not one

Step	Thread A	Thread B	counter in memory
1	load counter (5)		5
2		load counter (5)	5
3	add 1, giving 6		5
4		add 1, giving 6	5
5	store 6		6
6		store 6	6

Two increments, one update. In the next example 4 threads each add 1,000,000 times, and the total comes out well below 4,000,000, differently on every run.

PART 4 · MUTEXES · EXAMPLE 1 OF 4

A data race

Highlighted: counter++ is not atomic

```c
/* p4a_race.c - threads updating shared data without a lock.
   Build: gcc -Wall -pthread -o p4a_race p4a_race.c */
#include <pthread.h>
#include <stdio.h>

#define N_THREADS 4
#define N_ITERS   1000000

/* Shared by all threads. */
static long counter = 0;

static void *add(void *arg)
{
    (void)arg;
    for (int i = 0; i < N_ITERS; i++) {
        /* Not atomic: load, add one, store. Updates get lost. */
        counter++;
    }
    return NULL;
}

int main(void)
{
    pthread_t tid[N_THREADS];

    for (int i = 0; i < N_THREADS; i++)
        pthread_create(&tid[i], NULL, add, NULL);
    for (int i = 0; i < N_THREADS; i++)
        pthread_join(tid[i], NULL);

    printf("expected %d, got %ld\n", N_THREADS * N_ITERS, counter);
    return 0;
}
```

```
$ make p4a_race
$ ./p4a_race
expected 4000000, got 1304768
```

WHAT TO LOOK FOR

- 4 threads each add 1,000,000 times
- Expected 4000000, but updates are lost
- A different wrong answer on every run

PART 4 · MUTEXES

The mutex API

Call	What it does
PTHREAD_MUTEX_INITIALIZER	Sets up a global or static mutex at compile time
pthread_mutex_init	Sets up any other mutex at run time, before first use
pthread_mutex_lock	Waits until the mutex is free, then owns it
pthread_mutex_trylock	Takes it if free, otherwise returns EBUSY at once
pthread_mutex_unlock	Releases it; only the owner should unlock
pthread_mutex_destroy	Frees its resources once nobody uses it

Put every access to the shared data between lock and unlock, reads included, and keep the locked section short.

PART 4 · MUTEXES · EXAMPLE 2 OF 4

Protecting the counter

Highlighted: pthread_mutex_lock, pthread_mutex_unlock

```c
/* p4b_mutex.c - protect shared data with a mutex.
   Build: gcc -Wall -pthread -o p4b_mutex p4b_mutex.c */
#include <pthread.h>
#include <stdio.h>

#define N_THREADS 4
#define N_ITERS   1000000

static long counter = 0;

/* A mutex that is ready to use, set up at compile time. */
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

static void *add(void *arg)
{
    (void)arg;
    for (int i = 0; i < N_ITERS; i++) {
        /* Only one thread at a time gets past this line. */
        pthread_mutex_lock(&lock);
        counter++;
        /* Let the next waiting thread in. */
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main(void)
{
    pthread_t tid[N_THREADS];

    for (int i = 0; i < N_THREADS; i++)
        pthread_create(&tid[i], NULL, add, NULL);
    for (int i = 0; i < N_THREADS; i++)
        pthread_join(tid[i], NULL);
    printf("expected %d, got %ld\n", N_THREADS * N_ITERS, counter);
    return 0;
}
```

```
$ make p4b_mutex
$ ./p4b_mutex
expected 4000000, got 4000000
```

WHAT TO LOOK FOR

- Lock before touching shared data, unlock right after
- The total is now exactly 4000000 on every run
- PTHREAD_MUTEX_INITIALIZER needs no init call

PART 4 · MUTEXES · EXAMPLE 3 OF 4

Trying for a lock

Highlighted: pthread_mutex_trylock

```c
/* p4c_trylock.c - try for a lock without waiting.
   Build: gcc -Wall -pthread -o p4c_trylock p4c_trylock.c */
#include <errno.h>
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

static void *holder(void *arg)
{
    (void)arg;
    pthread_mutex_lock(&lock);

    /* Keep the lock for two seconds. */
    sleep(2);
    pthread_mutex_unlock(&lock);
    return NULL;
}
```

```c
int main(void)
{
    pthread_t tid;

    pthread_create(&tid, NULL, holder, NULL);

    /* Give the holder time to take the lock first. */
    sleep(1);

    /* Returns 0 if we got the lock, EBUSY if someone holds it. */
    if (pthread_mutex_trylock(&lock) == EBUSY) {
        printf("busy: lock is held, doing other work\n");
    } else {
        printf("got the lock\n");
        pthread_mutex_unlock(&lock);
    }
    pthread_join(tid, NULL);
    return 0;
}
```

```
$ make p4c_trylock
$ ./p4c_trylock
busy: lock is held, doing other work
```

WHAT TO LOOK FOR

- Returns at once instead of waiting
- 0 means you hold the lock, EBUSY means someone else does
- Useful when there is other work you could do

PART 4 · MUTEXES · EXAMPLE 4 OF 4

A mutex created at run time

Highlighted: pthread_mutex_init, pthread_mutex_destroy

```c
/* p4d_mutex_init.c - a mutex initialised at run time.
   Build: gcc -Wall -pthread -o p4d_mutex_init p4d_mutex_init.c */
#include <pthread.h>
#include <stdio.h>

/* The lock travels with the data it protects. */
struct account {
    pthread_mutex_t lock;
    long balance;
};

static void *deposit(void *arg)
{
    struct account *a = arg;
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&a->lock);
        a->balance += 1;
        pthread_mutex_unlock(&a->lock);
    }
    return NULL;
}
```

```c
}

int main(void)
{
    pthread_t t1, t2;
    struct account acct = { .balance = 0 };

    /* No static initializer here, so initialise it first... */
    pthread_mutex_init(&acct.lock, NULL);
    pthread_create(&t1, NULL, deposit, &acct);
    pthread_create(&t2, NULL, deposit, &acct);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("balance = %ld (expected 200000)\n", acct.balance);

    /* ...and destroy it when nobody uses it any more. */
    pthread_mutex_destroy(&acct.lock);
    return 0;
}
```

```
$ make p4d_mutex_init
$ ./p4d_mutex_init
balance = 200000 (expected 200000)
```

WHAT TO LOOK FOR

- PTHREAD_MUTEX_INITIALIZER for globals, init for the rest
- Keep the mutex next to the data it protects
- Destroy it only when no thread can still use it

WRAP-UP

What to remember

- Threads share memory; each keeps its own stack
- Join or detach every thread you create
- Give each thread its own argument, never the address of a loop variable
- Pass results back through pthread_join, on the heap
- Unprotected shared data is a race; protect it with a mutex
- Keep locked sections short; use trylock when waiting is not acceptable
- Initialise mutexes you do not set up statically, and destroy them after use

WRAP-UP

Where to go next

- Condition variables and the producer-consumer pattern
- Deadlock and lock ordering
- Read-write locks, spinlocks, barriers and semaphores
- Thread-local storage and thread-safe functions
- Cancellation and signals
- Debugging with gdb and ThreadSanitizer

Working programs for all of these are in the advanced folder of the zip.